中国科学技术大学

本科毕业论文



超图最小生成树算法-平推算法和在 RSMT 问题上的应用

作者姓名：	林靖凯
学　　号：	PB20061314
专　　业：	电子科学与技术
导师姓名：	王行甫 教授
完成时间：	2024 年 6 月 11 日

摘 要

本文主要工作是提出了用于解决超图最小生成树问题的新的思路和算法，并进行时间测试。

本文基于问题分解的思想，研究最小生成树在超图的一个部分上的所有可总数。用同型定理等减少状态总数。逐条加入边增大该部分来得到最小生成树。

基于此思想提出了 4 种具体的超图最小生成树算法：分部组合法、简单平推法、过滤法和匹配法。重点分析后 3 种算法并实现，并在一个特定类型的超图最小生成树问题——最小直角斯坦纳树问题的 GeoSteiner 算法中的计算超图最小生成树步骤——上进行时间测试。

测试结果发现本文算法效率远低于 GeoSteiner 当前使用的超图最小生成树算法，不过这一思路仍可能具有借鉴价值。

关键词：超图；超图最小生成树；最小直角斯坦纳树

ABSTRACT

This paper presents a new kind of way to solve the hypergraph minimal spanning tree (MST) problem, and tests its time cost.

Based on the idea of decomposition, we focus on all possible states of one part of the hypergraph, use the concept of state-form to reduce the number of states, and finally solve the problem by adding edges one by one to the part.

We present 4 concrete algorithms: Depart-and-Combine, Direct-Expand, Filter, Match. We focus on the latter 3 algorithms and realize them, and then test them on a specific type of hypergraph MST problem—hypergraph MST calculation step of RSMT algorithm GeoSteiner.

The result shows that this algorithm is far less efficient than the one GeoSteiner uses. Yet it could still be a new perspective.

Key Words: hypergraph; hypergraph minimal spanning tree; MST; Rectilinear Steiner Minimum Tree; RSMT; GeoSteiner

目 录

第 1 章 绪论 2
1.1 导读 2
1.2 超图最小生成树 2
1.3 RSMT 问题 2
1.4 GeoSteiner 算法 3
1.5 RSMT 超图的特殊性 3
第 2 章 基本理论 4
2.1 基本思想 4
2.2 奇异性假定 4
2.3 核心概念定义 4
2.4 状态组过滤 5
第 3 章 具体算法 8
3.1 分部组合法 8
3.2 简单平推法 8
3.2.1 状态分裂 8
3.2.2 整体流程 9
3.2.3 分裂操作 11
3.2.4 过滤操作 11
3.2.5 边集A实现 12
3.2.6 型K实现 12
3.2.7 时间分析 12
3.3 过滤法 14
3.3.1 流程 14
3.3.2 动态调整m阈值 15
3.4 匹配法 15
3.4.1 相交匹配过滤 15
3.4.2 模式网 16
3.4.3 其他 17
3.4.4 可选-模式扩展 18
3.4.5 可选-相交模式合并 18
3.4.6 可选-奇异度处理 18
第 4 章 实现测试 21
第 5 章 总结和展望 23
5.1 总结 23
5.2 展望 23
参考文献 24
附录 25
附录 1 同型定理扩展 25
附录 2 与其他算法思路比较 25
附录 3 用于其他超图 25
附录 4 推进顺序 26
致谢 27

第1章 绪论

1.1 导读

本文提出了求解超图最小生成树（1.2 节）的一种新的思路（第 2 章）和 4 种具体算法（第 3 章），并在 RSMT 问题上进行了时间测试（第 4 章）。

第 1 章介绍了相关背景。RSMT（Rectilinear Steiner Minimum Tree，最小直角斯坦纳树）问题（1.3 节）是一个平面布线问题。GeoSteiner（1.4 节）是 RSMT 问题的一个精确算法，GeoSteiner 算法的一个步骤是计算超图的最小生成树。本文提出了平推算法并关注其在这一特定类型的超图（1.5 节）上的表现。

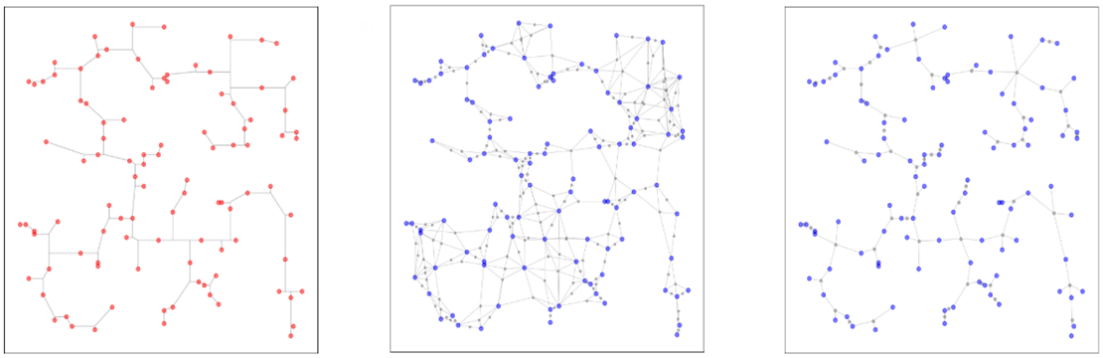
第 2 章介绍了本文算法的基本思想（2.1 节），定义了相关核心概念（2.3 节），并提出了减少算法时间的关键——同型定理（2.4 节）。

第 3 章介绍了基于此思想的四种具体算法，给出算法流程，分析一些实现中的细节，并探讨时间复杂度。

第 4 章是时间测试。

第 5 章是总结和展望。

附录补充了一些相关细节。



(a)RSMT 问题的解　(b)Fst 超图　(c)超图最小生成树

图 1

1.2 超图最小生成树

定义-超图：和普通的图类似，只不过一条边可能有多于 2 个端点。（见图 1b）

定义-连通：超图$G=(V,E)$，$E_0 \subseteq E$，当E_0有唯一的连通分量且覆盖V中的所有点时，称E_0或子图$G_0=(V,E_0)$是连通的。

定义-超图最小生成树 MST（minimal spanning tree）：带权超图$G=(V,E)$的最小生成树MST是G的权重最小的连通子图（可能有多个，当G本身不连通时不存在）。（见图 1c）

1.3 RSMT 问题

定义-RSMT 问题：给定平面上的点集V，用垂直和水平两个方向的一些直线将所有的点连通在一起（如图 1a，斜线视作呈 L 型的相连的一条水平线和一条垂直线）。求满足以上条件的线长度最小的布线方式。

RSMT 问题是个 NPC 问题[1]，因此精确算法的时间复杂度不太可能在多项式以内，往往是点集大小$n = |V|$的指数量级。

1.4 GeoSteiner 算法

RSMT 问题当前高效的精确算法是 GeoSteiner[2][3]，拥有成熟的软件[1]。算法分为两部分，第一部分得到一个超图，第二部分计算该超图的最小生成树。

第一部分先计算出一个充分的 Fst 集合（Fst 是将V中几个相邻点连接在一起的局部连线，具体参考文献[2]，充分指该集合包含组合得到最优连线必要的 Fst）。

第二部分要在第一部分得到的 Fst 集合中找到一个线长最小的子集连通所有的点。将每个 Fst 看作一个带权超边e（权是局部线长，连接点是 Fst 的连接点），就得到了超图$G = (V, E)$（图 1b），求解 Fst 集合的最小连通子集等价于求解G的最小生成树（图 1c）。以下称这样得到的G为 RSMT 超图。

第一部分的时间复杂度近（n的）线性，第二部分时间复杂度近指数，因此算法的时间瓶颈是第二部分。GeoSteiner 当前默认使用的超图最小生成树算法是一个高效的整数线性规划算法 branch-and-cut[4]。

1.5 RSMT 超图的特殊性

RSMT 超图有一些优于一般超图的特殊性质：

(1) **无环：**一般超图的最小生成树可能含有环，（如图 2，图中超图本身就是自己的最小生成树且有环）。但是由 RMST 问题的几何意义知 RMST 超图的最小生成树一定不会有环。

(2) **平面性：**由于 RSMT 超图是从一个几何平面生成的，因此具有和平面相关的一些特点。

(3) 超边数量和点的数量成正比，且单个超边连接的点数一般不会很多（大部分<10）。[2]

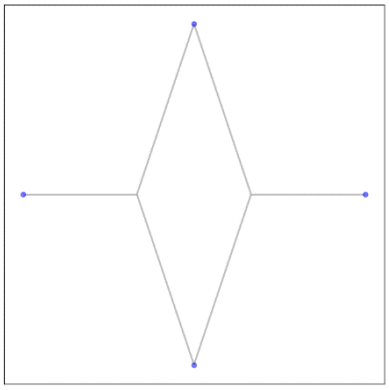


图 2 MST 有环的超图

本文的算法构建主要针对 RSMT 超图，利用了它的一部分特殊性质。稍微修改后（附录 3）也能用于一般的超图，但效率未知。

[1] http://geosteiner.com/

第2章 基本理论

2.1 基本思想

本文的算法基于“分部”思想，并且最初来源于这样的基本构思：将一个超图 G 分割为多个部分，如果能够对每个部分分别计算 MST，并且将结果组合起来后能得到 G 的 MST，那么这个问题的时间将会极大的减小（这是由于超图 MST 算法时间总是 G 大小的指数量级，将 G 分成几部分后时间指数量级地减少）。文献[5]就利用了这一思想将符合特定条件的最小斯坦纳树问题（不一定是直角）分割成几个子问题。当分割成功时这个方法的效率是很可观的，不过这一方法：

(1) 不是用于抽象超图，而是用于定义了具体度量的平面上的点集连线问题。

(2) 需要特殊的条件，并不总是有用，特别是对 RSMT 问题效果很有限。

本文的思想则从另一种角度，对超图最小生成树问题进行“不严格”的分割。即，任意取 G 的一部分超边，G 的 MST 在这一部分上拥有一个状态，我们无法严格计算出这个状态，但是能够计算出一个状态集合包含这个状态。于是 G 的最小生成树问题就变成了计算 G 的一部分的可能状态的问题。这样的分割方式不像严格的分割方式能明显提升效率（实际上这种方式只能独立成一种算法，甚至不好定义“提升效率”。而严格的分割方式因为分割后需要计算的是子部分的 MST，因此可以与其他 MST 算法组合使用），不过相比于严格方式需要的苛刻条件，具有更通用的特点。

出于时间复杂度考量，计算出各个部分的可能状态后，在实现中并不是通过“组合”各部分的方式得到 MST，而是通过一种“扩展（平推）”（3.2 节）的方式。

2.2 奇异性假定

为了分析方便，在接下来的理论分析中，在未特别提及时，默认超图的边权是奇异的。

定义-奇异性：称带权超图$G = (V, E)$是奇异的，是指

$\forall E_1, E_2 \subseteq E,\ E_1 \neq E_2 \Rightarrow w(E_1) \neq w(E_2)$

其中$w(E_1)$定义为E_1中边的权重之和。

也就是，如果E奇异，E的任意两个不同子集的权重不同。

在这一假定下，显然G的最小生成树是唯一的（因为不同树的权重不同）。对实际超图这一假定可能不正确，因此实现时需要在细节上重新考虑。

2.3 核心概念定义

定义-层：对超图 $G = (V, E)$，我们将E的任意一个子集 L 称作一个层。层就是我们所要考虑的G的“部分”。

定义-状态：层L的任意一个子集S称作L的一个状态（S必须在指定的L中才有意义）。状态就是层中边的具体选择方式。显然，层L的状态总数是$2^{|L|}$。定义状态的权重$w(S)$为S中边的权重和。

定义-解状态：对层L，将L上的状态$S_0 = MST \cap L$称作解状态或有效状态。也称S_0有效。这是我们希望求得的状态（特别的，当$L = E$时，$S_0 = MST$）。所有其他状态都可以称作无效状态。当超图奇异时一个层的解状态唯一。
定义-模式：一个层L和L上的一组状态$\{S_i\}$组成的结构 $M = (L, \{S_i\})$ 称作一个模式。
定义-有效模式，有效状态组：给定L时，我们难以直接求得有效状态S_0，往往只能计算得到一个状态组$\{S_i\}$包含S_0。这种状态组包含解状态S_0的模式$M = (L, \{S_i\})$称作有效模式，$\{S_i\}$称作有效状态组。

2.4 状态组过滤

考虑计算有效状态组的过程：已知层L，容易穷举得到L对应的全部$2^{|L|}$种状态，这其中自然包含正解，但是明显这样的状态数太多了。我们需要通过一些方法过滤掉一部分无效状态，来得到一个相对精简的有效状态组。
定义-剩余层L'：L的剩余层$L' = E/L$
定义-内点：只与L接触的点
定义-导点：同时接触L和L'的点
定义-外点：只与L'接触的点

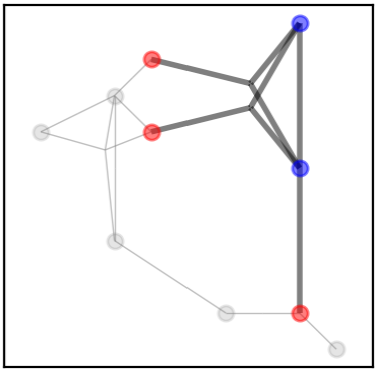


图 3 层L-深色边 剩余层L'-浅色边 内点-蓝色点 导点-红色点 外点-灰色点

以上 4 个概念由层L唯一确定，与具体状态无关。
我们基于以下三个条件对状态组进行过滤：

(1) 排除孤立状态

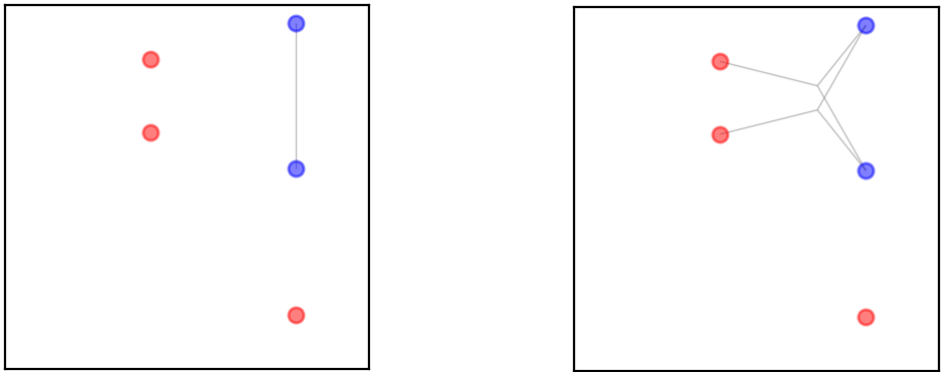
定义-孤立状态：当状态S存在一个连通分量不与L'相连时（即该连通分量只覆盖内点，不覆盖导点），称状态S孤立。

如图 4a，S有 4 个连通分量，其中三个是单独的点，另一个是孤立的。

当状态S孤立时，不论如何选择剩余层L'上的状态S'，$S \cup S'$都是不连通的，不可能形成MST，所以孤立的状态一定是无效的。

(2) 排除有环状态

当状态S有环时（如图 4b），S是无效状态（因为 RSMT 语境下超图最小生成树的无环性）。



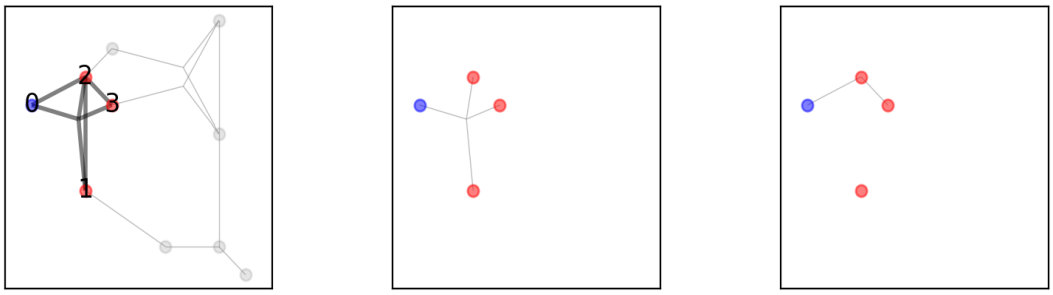
(a)孤立状态　　(b)有环状态

图 4 孤立状态和有环状态（层是图 3）

(3) 同型状态保留最小

定义-型：一个状态S的型，是所有导点在S中的互相连接状况，用导点的所有连通分量表示（图 5）。

同型定理：假设超图奇异，若有效状态组$\{S_i\}$中有多个状态型相同（图 6），可以只保留其中权重最小的那一个，删除其他的。这样过滤后的状态组仍然有效。



(a)层L　　(b)状态S_1的型是{{1,2,3}}　　(c)状态S_2的型是{{1},{2,3}}

图 5

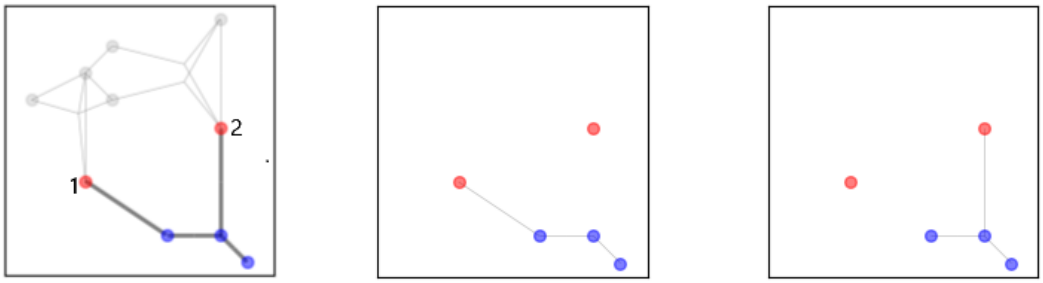


图 6 两个状态同型，都是{{1},{2}}

注：若超图不奇异，当型为K的所有状态中有多个最小等权重的状态时，可以随便保留最小状态中的一个。

该定理的直观解释如下。考虑$L' = E/L$中的连线方式，有一位连线者负责L'部分的连线（即对L'中的超边进行选择），目标是和L中的连线组合起来得到MST。型相同的两个状态在连线者的视角看来具有相同的连通性，即：

1.如果一个连线方式能连通一个状态，那么也能连通另一个状态。

2.如果一个连线方式不能连通一个状态，那么也不能连通另一个状态。

连线者要得到一个连通的 G 的子图，只需要将型中各个组相连，不需要考虑具体状态。因此，对连线者而言，型相同的两个状态的连通性相同，唯一的区别就是权重不同。既然如此，权重较大的那个状态就没有被保留的价值。

同型定理证明：

证明可以分为两部分：

a）同型的状态连通性相同。

其中，两个状态$S1$与$S2$连通性相同指：

对L'的任意一个状态S'

$S' \cup S1$连通 $\Leftrightarrow S' \cup S2$连通

从直观上容易看出，严格证明略。

b) 有效状态组中对连通性相同的状态只保留权重最小的一个，仍然有效。

参考附录 1。

第3章 具体算法

这一章介绍依据分部理论衍生出的四种算法：分部组合法、简单平推法、过滤法和匹配法。

其中分部组合法难以实现。后三种算法统称平推算法。

过滤法是简单平推法的扩展，匹配法是过滤法的扩展。

3.1 分部组合法

分部组合法是一种因为时间复杂度过高而不适合实现的方法。但是由于其思路较直接（也是本算法的原始构思来源），且有利于作为后面算法的参考，这里进行简单的介绍。

(1) 二分法：将 G 分解成大致相等的两部分（层），分别计算各部分的有效状态组然后组合得到 MST。先不考虑如何计算两个部分的有效状态组，光是最后组合的过程就需要m^2（设m是有效状态组的大小）的时间复杂度。

(2) 多分法：类似于二分法，是划分的数量不同。组合时若是两两组合仍需m^2的时间复杂度。

由于m本身近似是指数量级(3.2.7 节)，m^2过大，不利于实现。我们希望能有一个算法的时间复杂度大约是m的一次而不是高次，这就有了后面的简单平推法。

当然如果能找到一个更高效的组合方式，分部组合法也是可以考虑的方法。

3.2 简单平推法

3.2.1 状态分裂

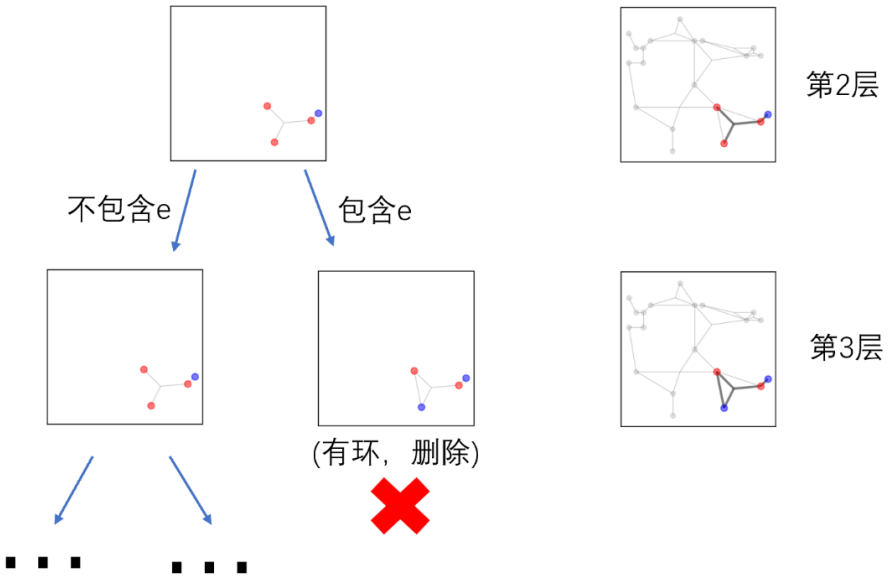


图 7 状态分裂和过滤

简单平推法从空层开始，逐条加入边扩大层直到$L = E$时完成求解。其基础是状态分裂：层L_1加入一条边e得到层L_2后，L_2的有效状态组的计算可以建立在L_1的有效状态组上。

理由是，L_1的有效状态组中一定包含解状态S_1，那么L_2上的解状态S_2在L_1中的部分一定是S_1，那么S_2只可能是S_1或$S_1 \cup \{e\}$两者之一。那么，如果将L_1的有效状态组$states1$的每一个状态 S 都按上面的方式分裂成S和$S \cup \{e\}$后加入$states2$，因为$S_1 \in states1$，这样得到的状态组$states2$包含S_1或$S_1 \cup \{e\}$，一定包含S_2，即是有效的。分裂后状态数变为 2 倍，此时可以通过 2.4 节的过滤条件过滤掉一部分状态。（状态分裂和过滤的过程见图 7）

3.2.2 整体流程

建立一个有效模式$M = (L, states)$。初始$L = L_{empty}$是空集，$states = \{S_{empty}\}$，S_{empty}是L_{empty}对应的空状态。

将G中的每条边e按照某种顺序依次加入L（图 8），使L不断扩大。加入边e后通过分裂和过滤状态来更新有效状态组$states$，并将其中连通的状态（也是连通子图）移出保存至一个连通状态表[2]。直到所有边加完，连通状态表中权重最小的就是目标解MST。

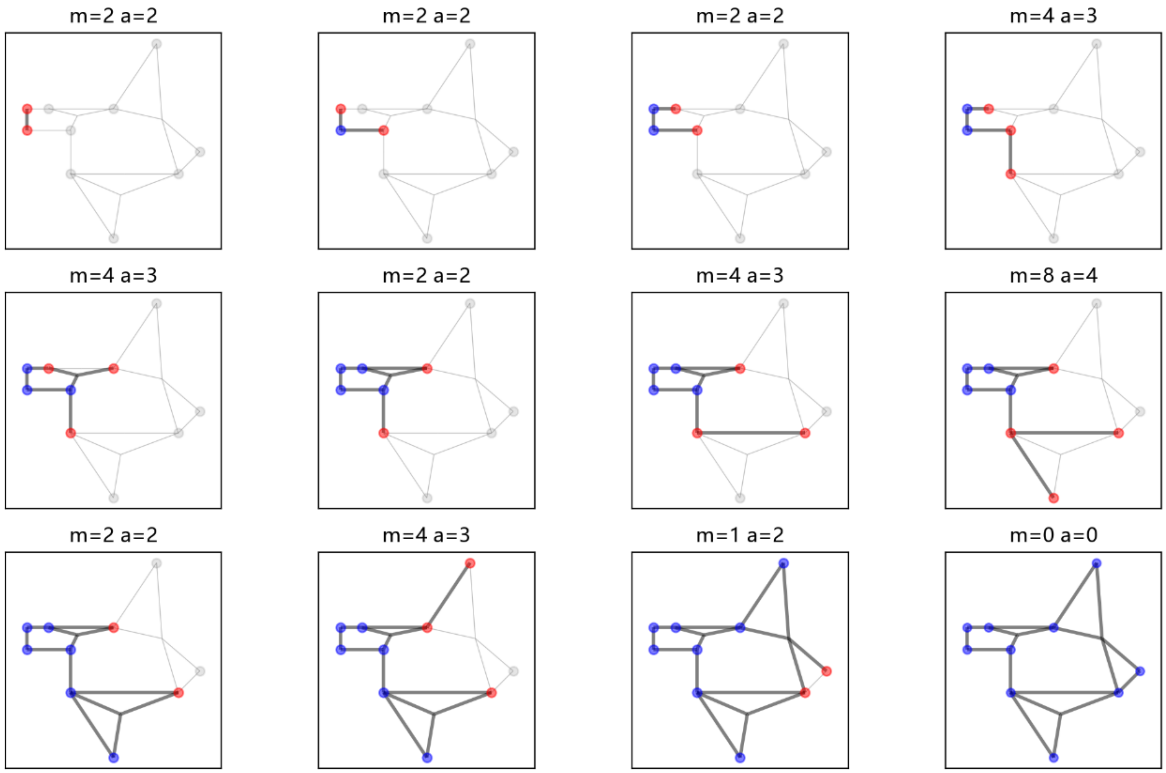


图 8 平推过程（黑边是L，m是有效状态数，a是导点数）

[2] 只需要求单个解时连通状态表可以只保存权重最小的一个

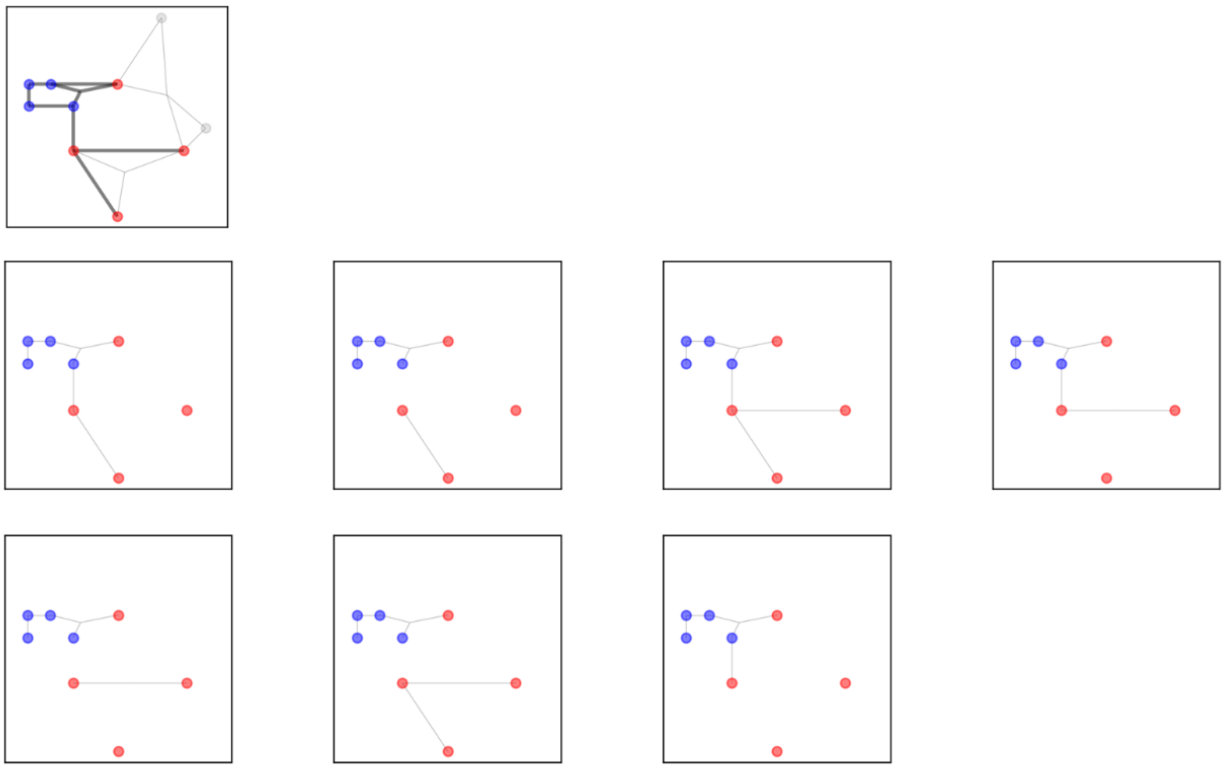


图 9 层和有效状态组

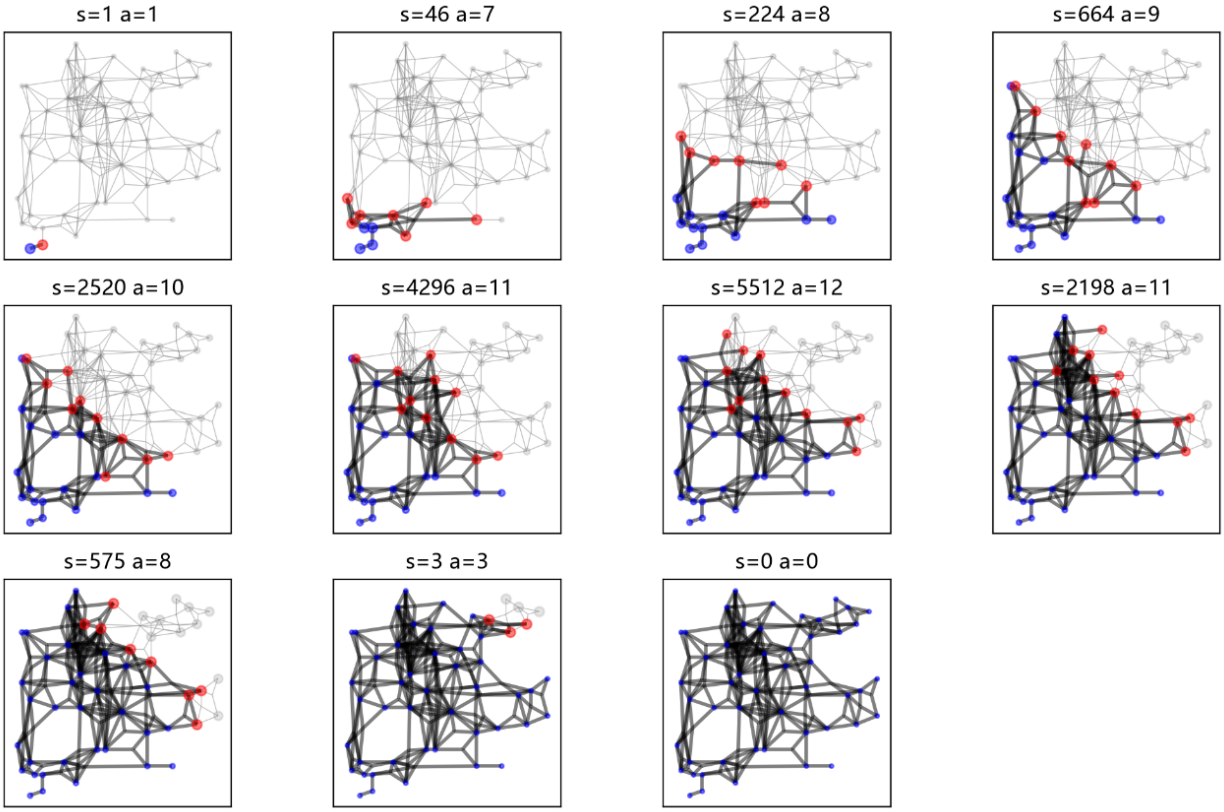


图 10 一个更大的简单平推法例子（没有显示每一边的加入过程）

简单平推算法的伪代码大致如下：

$states = [\ 初始空状态S_{empty}\ ]$
$L = 空层L_{empty}$
$finished_states = [\,]$ *#连通状态表*
依次对G中的每条边e：
    $L.add(e)$
    $states_new := [\,]$ *#新的有效状态组*
        对$states$的每个状态S：
            考虑S分裂后的两个状态S_0(包含e与否)，通过有环和孤立两个条件过滤后加入$states_new$。
    根据同型定理进一步过滤$states_new$中的无效状态。
    将$states_new$中的连通状态取出移入$finished_states$中。
    $states = states_new$
$finished_states$中权重最小的状态就是解MST。

单个状态S的数据结构是：$\{边集A, 型K\}$。

其中A主要用于求解到最后获取MST的结构，K用于过滤。如果只需要得到$|MST|$即总权重而不需要具体的超边集合，可以只实现K。

3.2.3 分裂操作

三个过滤条件中，前两个（有环和孤立）都可以在分裂前判断，所以S_0和S_1中只有没被这两个条件过滤掉的状态需要在内存中真正被分裂出来。因此每个S可能分裂出 0-2 个状态。

若S分裂成 0 个状态：不需要操作。

若S分裂成 1 个状态：需要更新S的数据。

若S分裂成 2 个状态：需要复制一个S，对两个S分别更新数据。

3.2.4 过滤操作

这一节介绍过滤状态组三个条件的具体实现方式。三个过滤过程都基于S的型K，与边集A无关。

(1) 有环：

当e的端点中有多于 1 个点位于S的型的同一个连通分量中时，S_1有环。
S_0不可能有环。

(2) 孤立：

定义-即将死去的导点：层 L 加入 e 后变为内点的导点。
定义-即将死去的外点：层 L 加入 e 后变为内点的外点。
若S的型的一个连通分量的所有导点都是即将死去的。或者e的端点中存在即将死去的外点。则S_0孤立。
若e的所有端点都是即将死去的，则S_1孤立。

(3) 同型：

利用同型定理对$states_new$进行的过滤的伪代码如下(字典结构参考 python 中的字典)：

建立字典$dict = \{$型K:状态$S\}$。
对每个状态$S \in states_new$，求出型K
　　如果K在$dict$中：比较S和$dict$的同型状态S'，在$dict$中保留权重较小的一个状态。
　　如果K不在$dict$中：将$\{K:S\}$加入$dict$。
$dict$中剩下的状态就是经过同型定理过滤后的状态。注意由于要用K作为字典的键，需要定义一个关于K的哈希函数$hash: K \to interger$。

3.2.5 边集A实现

边集A的实现方式有以下两种：
(1) 数组：用一个数组储存S的边。空间复杂度是$|L|$。复制数组需要$|L|$的时间。
(2) 树：用节点层数为$|L|+1$的二叉树储存当前所有有效状态（的赋值）。根节点代表空状态。第i层（$1 \le i \le |L|$，记根节点是第 1 层）的每个节点的左子节点代表第i条边不在 S 中，右子节点代表第i条边在 S 中。所有的叶子节点（在第$|L|+1$层）就是当前的有效状态组，每个状态的边集是从根节点到该叶子节点的路径。空间复杂度大约是$2m$。分裂时，只需要对所有叶子节点进行二分裂，时间复杂度是m。时空间复杂度平均到单个状态上都是常数。
用树实现的时空间复杂度更优。

3.2.6 型K实现

型K是所有导点的分割。使用以下字典结构实现：
　　$\{$*导点*n: *连通分量*$group\}$ # 每个导点到所属连通分量的查找字典
其中每个$group$是该连通分量包含的导点的集合。
单个状态空间为a。复制，哈希化的时间为a，查找（通过导点查找连通分量，或者通过连通分量查找导点）的时间为 1。

3.2.7 时间分析

(1) 状态数m估计

给定层L，我们希望估计出过滤之后的有效状态组$\{S_i\}$的大小m，m随推进过程而变化。在推进的最开始和最末尾的地方，m是最小的（图 11，只有唯一的状态）。随着推进进行，整体上m先增大后减小，在中间取得最大值 m_{max}。

m并不是一些参量的简单函数，只能通过已知条件进行估计。且多个因素有关：超边密度，导点数a，层的大小$|L|$等等。目前只能通过经验得知m可以用a的指数函数估计（底数近似为X，在区间(2,3)附近）。

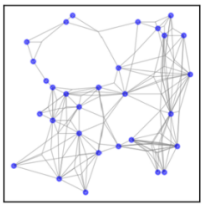
考虑近似正方形的G，在一条竖直直线从左向右水平扫描的过程中，a可以看作正方形的“边长”，因此$a \approx O(\sqrt{n})$，$m \approx O(X^a) \approx O(X^{\sqrt{n}})$。实际上随着$n$增大超边有可能在某部分变得特别密集，在较坏情况$m$下会比预期更大。

m的一个上限是L的所有型的数量$K(a)$，这是导点数a的一个精确的组合函数。由同型定理，每个型在有效状态组中最多只需保留一个状态，因此$m \le K(a)$。要用此上限估计m还需要知道$K(a)$的性质。

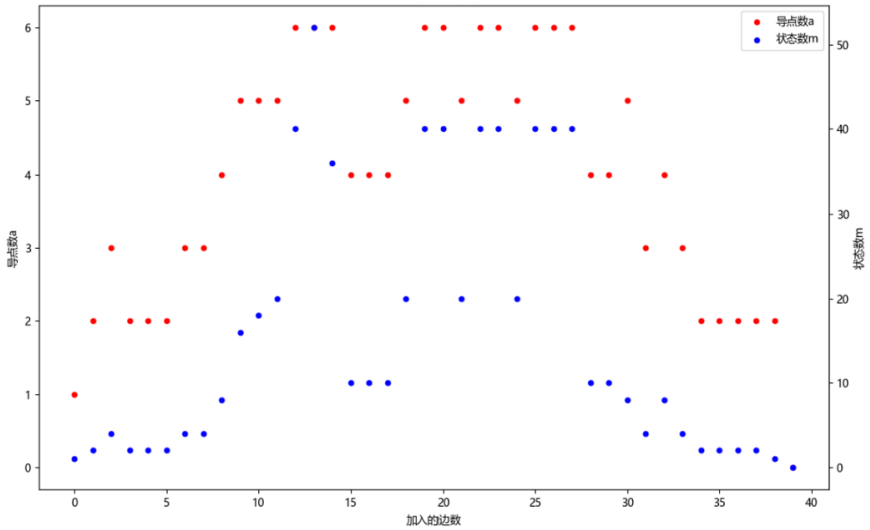
由于m对a的敏感度很高，要让m平均较小我们需要让a_{max}较小。这需要一个较好的推进顺序（见附录 4）。

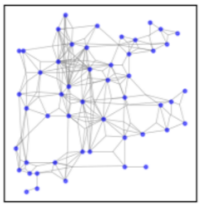
图 11 是两次简单平推法的状态数变化。可以粗略解读出以下几点：

（1）两个例子n相差不大m却差了上百倍，可以看出m指数爆炸的特点。
（2）m_{max}与$|E|$的相关性可能比与n的相关性更高。
（3）m和a的相关性很高，并且类似指数关系。
（4）m的峰值即算法的瓶颈集中在中间的某部分。



点数$n = 30$
边数$|E| = 40$
最大导点数$a_{max} = 6$
最大状态数$m_{max} = 52$





点数$n = 50$
边数$|E| = 124$
最大导点数$a_{max} = 13$
最大状态数$m_{max} = 14856$

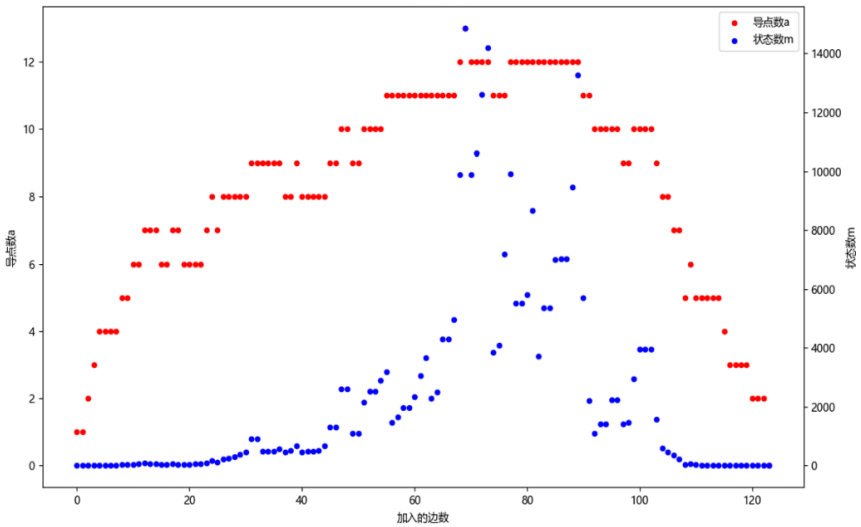


图 11 简单平推法的状态数

(2) 算法整体时间

设加入一条边e时对单个状态S操作花费的平均时间为t。

如果使用数组实现A，t的瓶颈是A的复制操作，$t = O(n)$。

如果使用树实现A，t的瓶颈是K的复制和哈希化，$t = O(a)$。

考虑到有$|E|$条边，设加入每条边时平均状态数是m，整个算法的时间是$m \times |E| \times t$。

考虑到m是指数量级而其他两项为多项式量级，增长速度上$m \gg t$，$|E|$。因此算法的时间复杂度主要取决于m。

3.3 过滤法

3.3.1 流程

过滤法在简单平推法的基础上做调整，能稍微改善简单平推法随n的增大指数爆炸的问题。

过滤法开始和简单平推相同，不同的地方是当遇到m指数爆炸的应对。

设置一个m阈值上限，当m达到或超过阈值时结束本次推进。这样的一次推进称作一次部分平推。

此时对得到的有效状态组进行分析，分别整理出在所有状态中都出现的边，记作集合E_1，以及在所有状态中都不出现的边，记作集合E_0。

易知E_1中的每条边一定会出现在 MST 中，E_0中的每条边一定不会出现在 MST 中。这两类边都可以从 G 中删除（删除方式不同），从而化简问题：

对$\forall e_0 \in E_0$，e_0一定不会出现在MST中，可以在G中删除e_0。

对$\forall e_1 \in E_1$：e_1一定会出现在MST中，可以将e_1退化为一个新的点n_1。

1. 删除所有与e_1有多于一个交点的边（因为MST的无环性）。
2. 所有与e_1有刚好一个交点的边，将该相交的端点修改为n_1。
3. 修改交点后得到的所有边中可能出现重叠的边，只保留其中权重最小的一个。
4. 删除e_1和e_1的所有端点。

经过过滤后，E变小了，重复部分平推和过滤，理论上有希望在m始终小于阈值的前提下完成MST的计算。

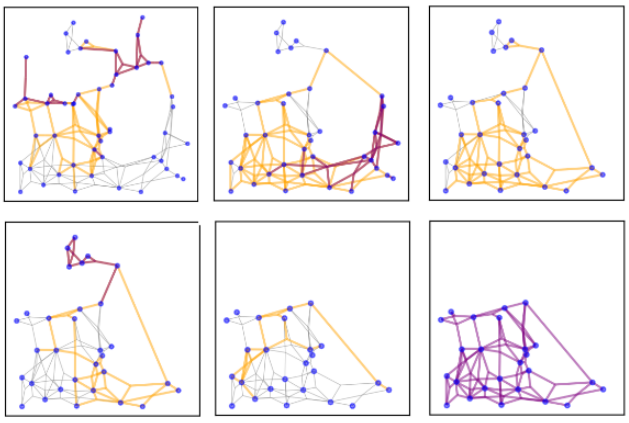


图 12 过滤法，黄色+紫色是每次部分平推的层，紫色是被过滤掉的边

但是实际上，在m阈值固定的前提下，能够被过滤的边其实只有一部分，即边较稀疏的部分（如图 13）。而对边较密集的部分时，在m达到阈值时，没法过滤任何一条边，要完成计算还需要提高m阈值。因此过滤法只能对简单平推法提供有限的改进。

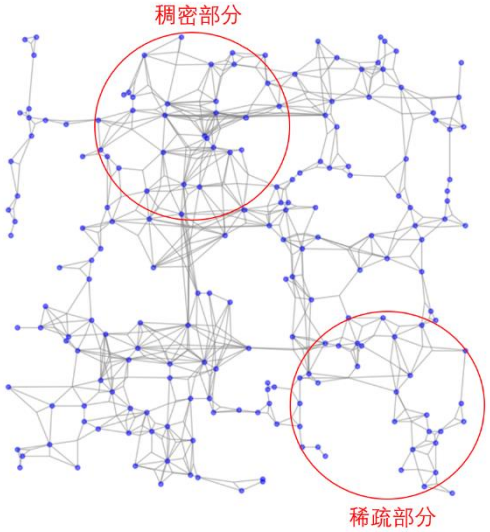


图 13 超图的稠密部分和稀疏部分

3.3.2 动态调整m阈值

相比于设定一个固定的m阈值，在过滤的过程中逐步增大m的阈值能提高效率，也能避免因为m阈值过小导致计算失败。一开始在超图有较多稀疏部分时，设置较小的m阈值进行快速过滤。在剩下的部分密度均较大时，只有增大m阈值才能完成求解。

可以用过滤失败的次数等参数作为触发m阈值增大的条件。例如，连续过滤失败 5 次后使m阈值翻倍。

3.4 匹配法

匹配法是基于过滤法的改进。我们注意到，过滤法中，每次部分平推的结果在过滤后会被抛弃。匹配法利用了这些原本会被抛弃的结果。

匹配法的基础和过滤法一样，分步过滤边逐渐减小超图。不同的地方在于，匹配法保留每次部分平推的模式，当两个模式相交时（层相交），可能可以过滤掉模式中的一部分状态。而模式变小能提高过滤掉超图中的边的可能性。

3.4.1 相交匹配过滤

当两个模式相交时，可能可以过滤掉模式中的一部分状态，原理简述如下。

当两个层相交时，两个层的有效状态组能够分别确定一个相交部分的有效状态组。相交部分中一个状态有效时它在两个层的有效状态组中都会出现。因此，如果相交部分一个状态只在其中一层的有效状态组中出现，那么该状态无效，可以被删除。其严格叙述如下。

定义-子状态：已知层L，$L_0 \subseteq L$，L的一个状态S，$sub = S \cap L_0$是L_0上的状态，称sub是S在L_0上的子状态。称sub属于S。

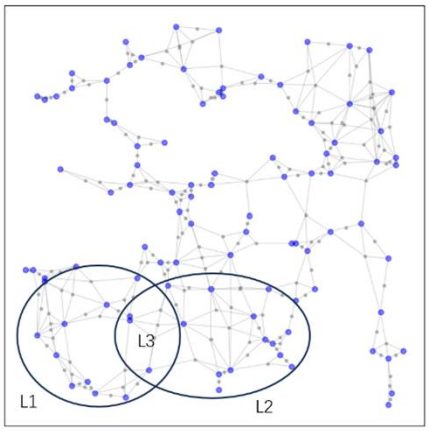
易得，sub无效 $\Rightarrow S$无效

S有效 $\Rightarrow sub$有效

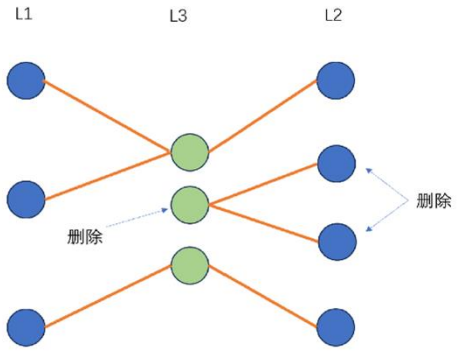
定义-子状态组：若对L上的状态组$states$中的每个状态S取S在L_0上的子状态sub，得到的所有子状态集合$subs$称作$states$在L_0上的子状态组。

易得，$states$有效 $\Rightarrow subs$有效

考虑两次部分平推得到的有效模式$M_1 = (L_1, states1)$和$M_2 = (L_2, states2)$，若层L_1和L_2有非空交集L_3，则称M_1与M_2相交（图 14a）。接下来说明两个模式相交时可以进一步过滤状态组。



(a)相交模式



(b)相交子状态和所属关系。蓝点是状态，绿点是子状态，橙线是所属关系

图 14

取$states1$在L_3上的子状态组$subs1$，$states2$在L_3上的子状态组$subs2$。$subs = subs1 \cup subs2$，注意$subs$中每个sub与M_1和M_2中的状态的所属关系（图 14b），每个sub属于M_1的 0 或多个状态，属于M_2的 0 或多个状态，并且sub不可能不属于任何状态。

定理：若某个sub属于M_1的 0 个状态，则sub无效， M_2中所有sub所属的状态也无效。（图 14b）（M_1和M_2是对称的，在定理中可以互换）

证明：

$states1$有效 $\Rightarrow subs1$有效

sub不属于M_1的任何状态 $\Rightarrow sub \notin subs1 \Rightarrow sub$无效

3.4.2 模式网

为了保存每次部分平推的模式，建立一个普通图，称作模式网（图 15）。每次部分平推得到的模式是网中的节点，若两个模式相交，对应的两个节点有一条边相连，每条边上的数据结构如图 14b。

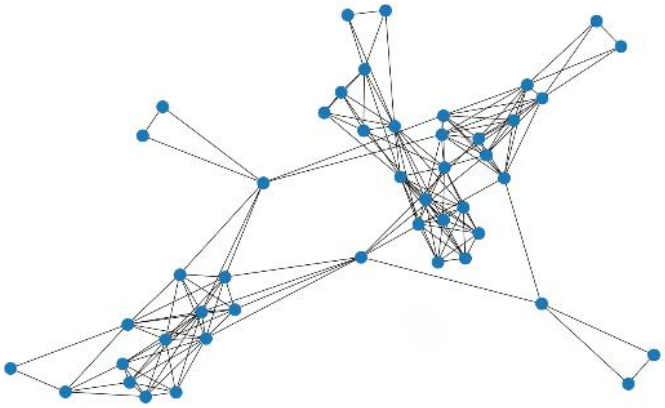


图 15 模式网

每次部分平推后，都在网中加入对应模式并与相交的模式连接，然后通过上述相交匹配原则进行状态过滤。伪代码如下：

操作-删除模式M的状态S：

对M的所有相交模式M'：

设sub是S在M与M'相交部分上的子状态。

删除S与sub的所属关系。

若sub在M中的所属状态数为 0：删除sub和sub在M'中的所属状态。

从上述流程可以看出，删除M的一个状态S，可能会导致相邻的模式M'的状态S'被删除，继而可能导致与相邻的模式相邻的模式M''的状态S''被删除……。因此，一个状态删除可能在网中引发连锁反应导致多个状态被删除。

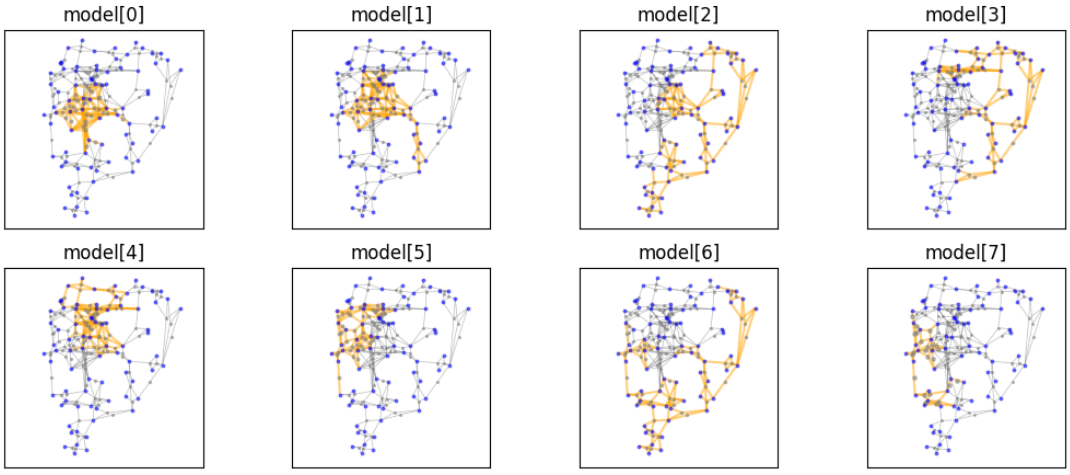


图 16 模式网中的模式层

3.4.3 其他

(1) 动态调整m阈值

除了在 3.3.2 节中提到的连续过滤失败的次数，触发m阈值增大的条件还可以是当前模式网的大小，覆盖率（平均每条边被包含的模式数量）等。

(2) 过滤更新模式网

每次过滤之后因为超图改变了，需要对模式网中的模式进行更新，实现起来有点复杂。
也可以选择清空模式网重新开始，但是效率会降低。

3.4.4 可选-模式扩展

除了每次从空模式开始部分平推，在模式网加入新模式，也可以对模式网中已有的模式进行扩展（即继续加入边直到m达到阈值）。
注：被扩展的模式m值之所以小于阈值，有可能是因为相交匹配过滤掉模式中的部分状态，也有可能是m阈值动态增大。

3.4.5 可选-相交模式合并

在分部组合法中，两个状态数都接近m的模式合并会导致m^2的时间复杂度。

但是对两个相交的模式，可以通过子状态进行合并（两个状态可以合并的前提是相交部分的子状态相同）。如果每个子状态连接的两边的状态都较少，则只需要一个m的较小倍数的时间来进行合并操作：

操作-模式合并：
对每个子状态，组合两边的关联状态。
根据三个有效状态组过滤条件进行过滤。

在实现中，可以通过子状态的连接数计算每两个模式相合并的预估时间，当时间小于某个预设值时合并两个模式。

3.4.6 可选-奇异度处理

3.4.6.1 奇异度概念

定义-奇异度：类比取布尔值的奇异性，我们将奇异度模糊地定义为一个量化值：一个超图$G = (V, E)$ 的边集E有越多不同子集的权重和相同，就认为这个超图的奇异度越弱。

一般来说，点的横纵坐标的精度越高G的奇异度越强。

如坐标为高精度浮点数时G一般拥有较大的奇异度，坐标为范围较小的自然数时G一般拥有较小的奇异度。

3.4.6.2 奇异度影响

一个超图的奇异度对简单平推法和过滤法的效率无影响，对匹配法的效率有影响。

设型K，有效模式$M = (L, states)$，$states$中型为K的所有状态为集合$states0$，$states0$中权重最小的所有状态为集合$states0_min$，一般奇异度越高$|states0_min|$越大。

对简单平推法和过滤法，$states0$只需随便保留$states0_min$中一个就能保证状态组仍然有效，效率与$|states0_min|$无关，因此与奇异度无关。

但是对匹配法，$states0$中，必须保留全部$states0_min$。因为在超图不奇

异时，可能存在多个MST，层L有多于一个解状态。若还是只保留$states0_min$中的某一个，则有效状态组可能只包含所有解状态中的一部分。

因此在相交匹配过滤时，$states1$包含的解状态集和$states2$包含的解状态集可能没有交集，$subs1$包含的解状态集与$subs2$可能没有交集，相交匹配过滤时把$states1$中的所有解状态都过滤掉使M_1无效，最终导致MST无解。

注意，保留全部$states0_min$后，匹配法最后会求出G的所有MST。

匹配法中超图奇异度较弱时，会导致计算时间较严重的增大（一般数倍）。一是$states0_min$中状态全部保留导致m增大，二是局部解的多样性导致更难确定某条边一定存在或某条边一定不存在，增加过滤难度。

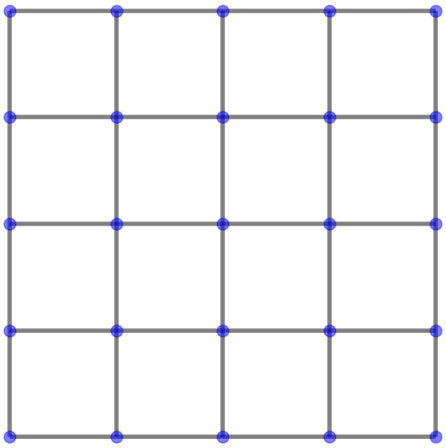


图 17 奇异度很低的超图

一个极端的例子如图 17（普通图是特殊的超图），点集$V=\{(i,j)：i,j\in\{0,1,2,3,4\}\}$，端点分布过于整齐，每条边长度（权重）相同，奇异度很低，因此：

(1) 每条边都可能存在也可能不存在，完全无法局部过滤掉任何一条边。
(2) 局部同型最小状态很多，m 很大。
(3) 正解数量过多（每一个连通树都是正解），明显无法在较短时间内全部计算出来，所以用匹配法很难计算出来。

3.4.6.3 奇异度处理方式

以下介绍两种增加超图奇异度的方式：

(1) 浮动法

对每个边权增减一个较小的高精度随机浮动值，在保证这点浮动不会导致正解变动的前提下，使边权变得更奇异。这种方式特别适用于边权都是整数（精度固定的浮点数，可以通过移位变成整数）的情况。此时当所有浮动值的绝对值之和小于 1 时（证明略），不影响正解。

(2) 排序法

事先取E的一个排序$order$。依据以下步骤比较任意两个边集E_1和E_2的权重：

函数-比较两个边集E_1和E_2的大小：

若E_1的边权之和与E_2的边权之和不同：返回两者的比较结果。

依次对$order$里的每个e：

若e只存在于E_1和E_2中的一个：返回包含e的那个边集权重更大。

返回$w(E_1) = w(E_2)$。*#此时$E_1 = E_2$*

以上步骤能保证任意E_1, E_2的权重和不同，即超图是奇异的，目标MST解也变得唯一（是原本的多个解中的一个）。

以上是排序法的理论判断方式，时间复杂度为$|E|$过大。实际实现中希望E_1和E_2是事先按照$order$排好序的，E_1和E_2的边权和事先计算出，按照以下流程比较E_1和E_2的权重：

函数-比较两个边集E_1和E_2的大小：

若E_1的边权之和与E_2的边权之和不同：返回两者的比较结果。

对$i = 1,2,3$ .. ：

取E_1的第i条边e_{1i}，E_2的第i条边e_{2i}。

若$|e_{1i}| < |e_{2i}|$：返回$w(E_1) < w(E_2)$。

若$|e_{1i}| > |e_{2i}|$：返回$w(E_1) > w(E_2)$。

若$|e_{1i}| = |e_{2i}|$：

若E_1，E_2均有第 i+1 条边：继续迭代。

若E_1，E_2均没有第 i+1 条边：返回$w(E_1) = w(E_2)$。

若E_1有第 i+1 条边，E_2没有第 i+1 条边：返回$w(E_1) > w(E_2)$。

若E_2有第 i+1 条边，E_1没有第 i+1 条边：返回$w(E_1) < w(E_2)$。

这种判断方式和上面的理论判断方式效果是相同的，但速度更快。函数在E_1和E_2的第一处不同处返回，时间复杂度不会超过$\min(|E_1|, |E_2|)$，实际上一般更小。

每个状态的边集A需要保持有序，不方便使用树实现，只能使用有序数组实现，加入边时需要有序插入。

因此这种方式会在一些步骤上增加时空间复杂度（见 3.2.5 节），实用性未测试。

第4章 实现测试

本文用 python 语言实现算法[3]。V是在正方形区域内随机均匀分布的点集。

事先用 GeoSteiner 求出超图[4]。注意 GeoSteiner 可以生成两种超图，一种是修剪过的，一种是未修剪过的（边数量是修剪过的数倍）。这里的测试使用修剪过的超图。如果使用未修剪过的超图，由于超边密度变大，m很大，计算时间会显著增大。

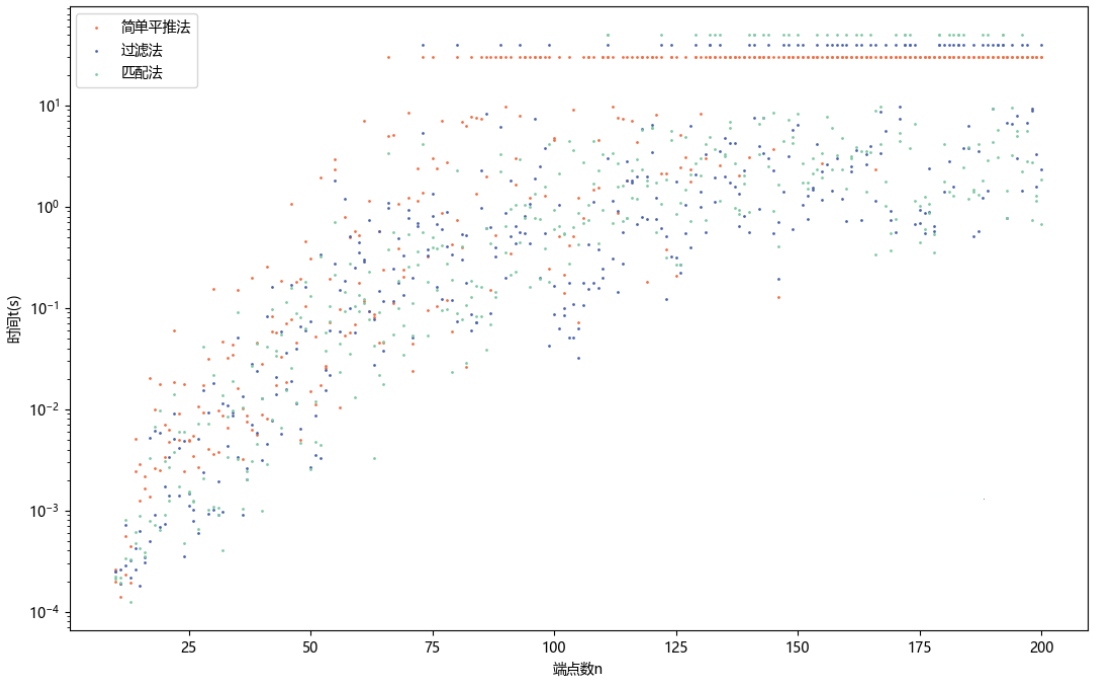


图 18 测试结果（最上面三条直线上的点代表超时，$t > 10s$）

我们对三种平推法进行了时间测试（图 18），时间t仅是计算 MST 所用时间，不包含 GeoSteiner 生成超图的时间。设置超时时间为 10s（超时后停止计算并记录为“超时”）。从图中可以看出：

(1) 时间t与具体问题相关性很大。n相同时t也可能摇摆两个数量级。一些困难的例子计算很慢。

(2) 三种平推算法比较。过滤法相比简单平推法有较明显的进步（从超时的部分容易看出），而匹配法较过滤法有不明显的进步。

(3) 与 branch-and-cut 比较。在 10s 内，简单平推法能基本稳定计算出结果的n上限大约是 75。过滤法和匹配法的上限大约是 125。而 branch-and-cut 算法

[3] 性能信息
处理器: 13th Gen Intel(R) Core(TM) i7-1360P 2.20 GHz
机带 RAM: 32.0 GB
设备 ID: 3E5ADC90-A102-4FEC-A291-E5F190CA4546
产品 ID: 00342-30929-87819-AAOEM
操作系统： Windows 11
编程语言：python 3.12.1

[4] GeoSteiner5.3
命令 rfst | prunefst
prunefst 代表修剪，详情见 GeoSteiner 手册。

计算 1000 个点[5]的时间是 2.3±1.4s（见[3]Table 3）。从这些数据可以看出，这里实现的平推算法的效率远低于 branch-and-cut(考虑到这是个指数复杂度算法，点数差 900 的差距是很大的)。即使使用更高效的编程语言（如 C 语言）实现并在细节上对算法进行优化，缩小常数倍的时间，预计比较结果不会改变。

[5] 这里直接引用文献[3]的数据，和本文使用的编程语言和机器环境不同

第5章 总结和展望

5.1 总结

本文实验结果表明，平推算法能够有效计算边数不过分多的超图最小生成树。但是相比于 branch-and-cut 算法，目前看来平推算法没什么实用价值，除非未来能有关键的改进。

5.2 展望

以下是本文算法的一些可能有意义的研究方向：

(1) 使用更高效的语言（如 C 语言）实现算法并对细节进一步优化。
(2) 结合使用过滤法和 branch-and-cut。先使用过滤法缩小超图大小，再用 branch-and-cut 计算出最小树。
(3) 对其他类型的超图进行测试。
(4) 过滤法和匹配法都有较多需要调节的策略和参数，可以在这一方面进一步优化。
(5) 对算法时空间复杂度进行更精确的理论分析（本文的分析主要基于经验），特别是过滤法和匹配法（本文没有分析）。
(6) 寻找更高效的组合方式，将分部组合法的思想应用到实现中。

参 考 文 献

[1] Garey, M. R., & Johnson, D. S. (1977). *The Rectilinear Steiner Tree Problem is NP-Complete. SIAM Journal on Applied Mathematics, 32(4), 826–834.* doi:10.1137/0132071

[2] Warme, D. M., Winter, P., & Zachariasen, M. (2000). Exact Algorithms for Plane Steiner Tree Problems: A Computational Study. Combinatorial Optimization, 81–116. doi:10.1007/978-1-4757-3171-2_6

[3] Juhl, D., Warme, D. M., Winter, P., & Zachariasen, M. (2018). The GeoSteiner software package for computing Steiner trees in the plane: an updated computational study. Mathematical Programming Computation. doi:10.1007/s12532-018-0135-8

[4] Warme, D.M. (1998) Spanning trees in hypergraphs with applications to Steiner Trees [Preprint]. doi:10.18130/v3zg4b

[5] Cockayne, E. J., & Hewgill, D. E. (1986). *Exact computation of Steiner minimal trees in the plane. Information Processing Letters, 22(3), 151–156.* doi:10.1016/0020-0190(86)90062-1

附 录

附录1 同型定理扩展

同型定理拥有一个理论上更强，但是难以有效实现的版本。
设超图$G = (V, E)$是奇异的。

型过滤定理：

已知$L \subseteq E$，$L' = E/L$。
已知L上的有效状态组$states$，L'上的有效状态组$states'$，$S \in states$。
若对$\forall S' \in states'$
$S' + S$连通 $\Rightarrow \exists S_1 \in states$，$S_1 + S'$连通，$w(S_1) < w(S)$
那么S是无效状态

证明：

反证：假设S有效
$\exists\, S' \in states'$，$S' \cup S = MST$连通
取出定理条件中的S_1，$S' \cup S_1$连通
$w(S_1) < w(S)$
$\Rightarrow w(S') + w(S_1) < w(S') + w(S)$
$\Rightarrow w(S' \cup S_1) < w(MST)$，与$MST$定义矛盾
所以S无效。

若两个状态S与S_1同型，$w(S_1) < w(S)$，用S和S_1分别替换上面定理中的S和S_1，就得到了同型定理。

附录2 与其他算法思路比较

(1) Backtrack Search

backtrack-search[2]是一种直接但效率不高的超图最小生成树算法。backtrack-search 可以看作是一种“深度搜索”算法（同一时间只考虑一个状态），相对的本文的平推算法可以看作一种“广度搜索”算法（同一时间考虑多个状态，即有效状态组）。

(2) Branch and Cut

branch-and-cut 算法[4]是 GeoSteiner 当前使用的超图最小生成树算法。计算过程可以概括为分支（branch）和剪切（cut）节点（[4]Figure 4.7，可以和本文图 7 作比较）。平推算法也同样可以看成是分支（分裂状态）和剪切（过滤状态）的过程（图 7）。
因此这两种算法在整体结构上有相似之处，不过理论基础不同：branch-and-cut 分支剪切的节点是限制条件，平推算法分支剪切的是状态。

附录3 用于其他超图

将状态组过滤条件中的“有环”去掉，本算法即可用于一般超图。但是效率未知。

理论上本算法也能用于 GeoSteiner 对其他类型平面最小斯坦纳树的计算

（具体类型见[2]），并且效率应该和 RSMT 超图差不多。

附录4 推进顺序

简单平推法中我们要确定E的一个排序来推进。在过滤法和匹配法中，由于要进行多次的部分平推，需要为每一次部分平推选择一个推进起点和推进顺序。好的推进顺序在层的大小一定的情况下能降低a，即降低m，即降低时间复杂度（或者m阈值确定时增加最终模式层的大小）。

一个较好的推进顺序要求选择的下一条边是当前已选择边集的相邻边，即保证连通分量总是唯一的。在此基础上还需要一个选择算法来判断在所有邻边之中选择下一条边。以下介绍几种思路：

a) 直线扫描

用一条垂直线从左向右扫描（或其他方向，具体细节自定义）。

这个方法实现简单，但是有以下缺点：

1) 使用了点的平面坐标值，在一般的抽象超图中无法使用。
2) 只能从边界开始，起点和顺序有限，只适用于简单平推法或过滤法，匹配法一般需要灵活的起点和顺序选择。
3) 对a_{max}的优化较差，特别是形状不规整的G。

b) 依据某种优先级函数f选择周围的下一条边

如$f(L,e)$ = *加入e之后层L的导点数量a的变化量*（f值越小优先级越高）。

c) 广度搜索

和普通图的广度搜索类似，具体细节自定义。

致 谢

在中国科学技术大学本科四年的学习既波折又充实，我在学到了丰富的知识的同时增长了不少人生阅历。在此感谢学校为我们学生提供的舒适的学习生活环境，感谢班主任尹东老师和同学们在我遇到问题时提供的帮助，感谢王行甫导师为我第一次撰写论文提供指导。望以此作为未来学术生涯的良好开端，继续努力。

2024 年 6 月